Production ML Churn Prediction

Technical project report | Model evaluation, serving, and monitoring

Project purpose. Demonstrate a practical machine-learning lifecycle for predicting customer inactivity: temporal dataset construction, model comparison, model registry, API inference, monitoring, and release review.

Important scope statement. The dataset is synthetic. Reported metrics are evidence about this project's dataset and implementation—not a claim of real-world business impact or live production deployment.

Executive summary

The project builds weekly customer snapshots and predicts whether a customer becomes inactive over the next 30 days. A time-based split and 30-day label embargo are used to reduce temporal leakage. Logistic Regression, Random Forest, and XGBoost were evaluated; Random Forest remains the champion because XGBoost did not meet the pre-defined meaningful-improvement gates.

The repository also includes a FastAPI serving layer, MLflow tracking and model registration, Prometheus instrumentation and alert rules, offline rolling-performance analysis, and drift checks. These components form a portfolio-grade implementation; live production readiness depends on environment-specific validation and operational integration.

1. Problem and dataset

Target: **inactive_next_30d**, a binary label for whether the customer is inactive over the following 30 days according to the dataset's label construction.

Dataset property	Value
Snapshot rows	3,239,195
Unique customers	48,704
Weekly snapshots	126
Snapshot range	2024-04-01 to 2026-08-24
Positive label rate	14.4%
Feature columns in full dataset	82 (including IDs, timestamp, target)
Train / validation / test	1,654,654 / 341,513 / 939,500 rows

The dataset contains customer and historical aggregates for orders, payments, support, subscriptions, marketing, and feedback. The verified customer-snapshot key is (customer_id, prediction_timestamp), with no duplicate keys in the inspected dataset. Customers can recur across snapshots; this is a temporal split, not a customer-disjoint split.

The held-out test period is 2026-04-06 through 2026-08-24. Its purpose is final evaluation and descriptive time-window analysis, not repeated model selection.

2. Model results

Validation metrics

Model	ROC-AUC	PR-AUC	Brier ↓
Logistic Regression	0.686180	0.238063	0.116942
Random Forest	0.687386	0.238627	0.116970
XGBoost	0.686406	0.235758	0.117036

Random Forest is retained as champion. XGBoost passed the configured safety gates but did not achieve either meaningful-improvement criterion. The metric differences are small, so this is a conservative gate-based decision—not a claim of broad model superiority.

Held-out Random Forest test metrics

Metric	Test result
ROC-AUC	0.690006
PR-AUC (average precision)	0.237343
Brier score	0.115705

ROC-AUC summarizes ranking discrimination; PR-AUC (average precision) is informative for the imbalanced positive class; Brier score measures probability error, with lower values preferred.

Historical monitoring sample — not a final test result

The 5,000-row historical ledger demo reported ROC-AUC 0.75162, PR-AUC 0.35004, and Brier 0.11318. This sample has a different temporal composition and is not comparable to the held-out test metrics. It demonstrates the monitoring workflow only.

3. System architecture

Offline data and training: processed weekly customer snapshots → time-based train/validation/test splits → preprocessing/model pipelines → MLflow experiment tracking and model registry.

Model selection: registered champion and candidate are evaluated on the validation split with explicit non-regression and meaningful-improvement gates. The current Random Forest v1 remains the champion; XGBoost v3 is not promoted.

Serving: a FastAPI service loads the deployment model bundle, validates request features, exposes health/prediction/metrics endpoints, and exports Prometheus metrics. Docker Compose defines local services.

Operations: Prometheus configuration and alert rules cover service signals. Offline scripts build a historical prediction ledger, match/mature labels from existing historical outcomes, calculate rolling metrics, and inspect drift.

Workflow overview

```
Processed snapshots → time-aware split → model training & MLflow → validation gates → selected model bundle → FastAPI → Prometheus/Grafana and offline performance/drift reports
```

4. Monitoring observations

Signal	Observed result	Interpretation
tenure_days PSI	≈ 0.432	High drift signal; investigate data/population changes
marketing_events_30d PSI	≈ 0.192	Moderate drift signal; investigate
Prediction-score PSI	≈ 0.004	Low observed score-distribution drift

Drift is a diagnostic signal, not proof of model degradation and not an instruction to retrain automatically. Rolling analysis used overlapping 60-day windows advanced weekly; because these windows overlap, they are not independent estimates.

The historical label-maturation exercise uses labels already present in the historical dataset and simulates availability after the outcome window closes. A live system would require actual outcome events, late-arriving-data handling, and an explicit data-completeness policy.

5. Promotion gates and release safety

Configured candidate gates: ROC-AUC regression no worse than 0.005; PR-AUC regression no worse than 2%; Brier worsening no greater than 5%; plus either ROC-AUC improvement of at least 0.002 or PR-AUC improvement of at least 2%. The current comparison decision is **DO_NOT_PROMOTE**.

A candidate passing these metric gates would only be eligible for a release review. Before deployment, verify schema compatibility, artifact identity, API behavior, segment performance, and rollback readiness. The held-out test set must not be reused for tuning promotion criteria.

6. Limitations and next steps

- Synthetic data may contain generator artifacts; segment effects and declining subscription coverage deserve scrutiny.
- Repeated customer snapshots mean splits are temporal but not customer-disjoint.
- Historical ledger metrics and held-out test metrics serve different purposes and must not be conflated.
- Drift and promotion thresholds are initial engineering defaults and need calibration against business costs and operating conditions.
- Before claiming production readiness, add automated tests for monitoring scripts, release/rollback verification, and environment-specific operational checks.

Repository entry points

src/models/ — model training, evaluation, registration and champion/candidate comparison.
src/api/app.py — FastAPI inference service.
monitoring/ — Prometheus configuration and offline ledger, rolling-performance, and drift utilities.
docs/model-evaluation/ — metric reports, ablations, threshold analysis, feature importance, and model comparison output.

This report describes the project state based on recorded metrics and implementation notes. Before publication, verify commands, paths, dependency versions, and CI jobs against the exact final repository.